

MINDANAO STATE UNIVERSITY - ILIGAN INSTITUTE OF TECHNOLOGY

College of Computer Studies
Department of Computer Applications
BCA182 Embedded Systems Programming

LABORATORY ACTIVITY NO. 1

Real-Time Multisensor Room Monitoring System

Name	Sherie Grace L. Lupo
Subject/Section	BCA182 B187
Date	September 28, 2026
Development Platform	STM32 Blue Pill + FreeRTOS + Wokwi

Laboratory Report

1. Problem and Requirements

This laboratory activity focused on the design and implementation of a real-time multisensor room monitoring system using the STM32 Blue Pill, STM32Cube HAL, FreeRTOS, PlatformIO, and Wokwi. The system monitors temperature, humidity, ambient light, and motion, while also providing user interaction through a rotary encoder and visual feedback through an SSD1306 OLED.

The simulated hardware includes a DHT22 sensor for temperature and humidity, an LDR for ambient light, a PIR sensor for motion, a rotary encoder for display navigation, an OLED for presenting measurements, and a buzzer for temperature alarms. The required temperature limits are 18 °C for the lower threshold and 30 °C for the upper threshold.

The system was required to behave as a concurrent embedded application rather than placing all behavior in a single task. FreeRTOS tasks, queues, an event group, a mutex, periodic delays, and an ACTIVE/INACTIVE state machine were therefore used. If no motion is detected for about 15 seconds, the system becomes INACTIVE. Motion detected by the PIR sensor returns the system to ACTIVE.

Wokwi was used to adapt the project to a simulated environment. This allowed sensor values to be changed manually and system behavior to be observed without physical hardware.

2. System Architecture and Design

The system is divided into sensor input, application processing, inter-task communication, and output subsystems. SensorTask handles the DHT22 and LDR. MotionTask monitors the PIR sensor. InputTask handles the rotary encoder. StateTask manages the ACTIVE and INACTIVE state. DisplayTask owns the OLED, while AlarmTask evaluates temperature conditions and controls the buzzer.

System Architecture

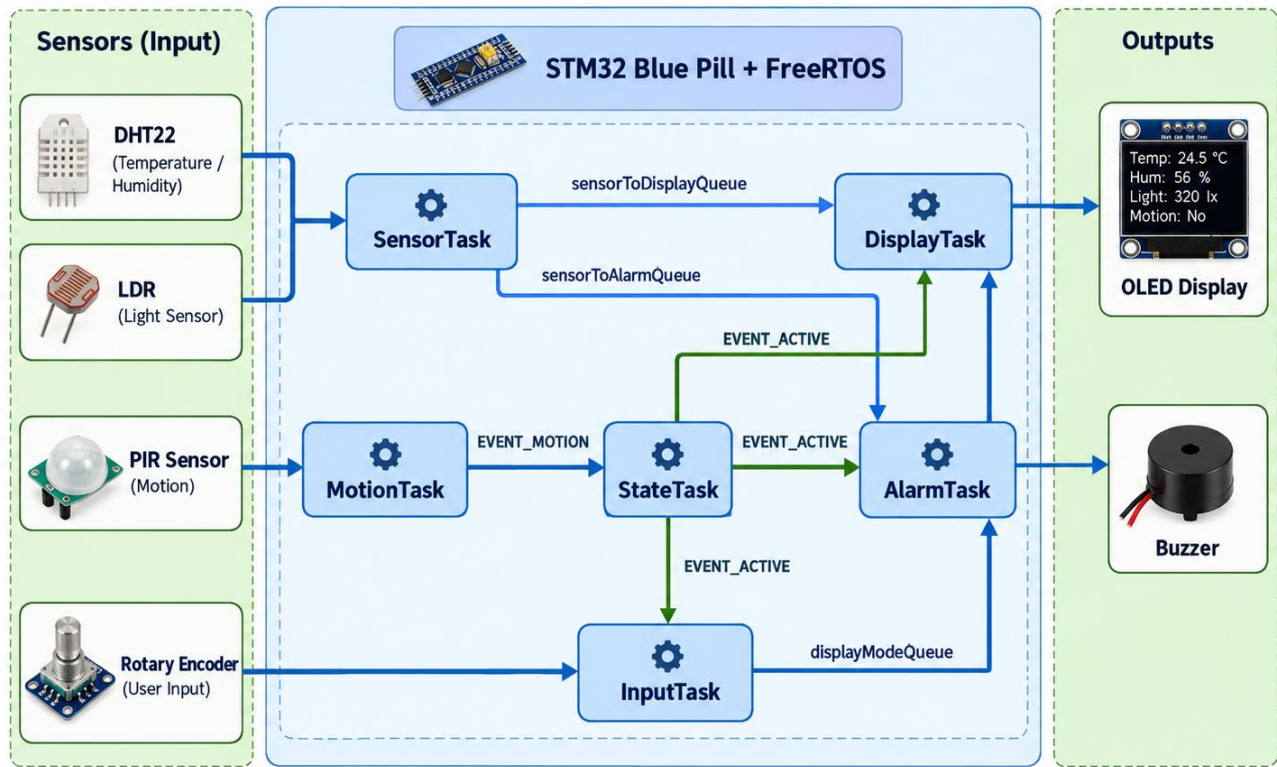


Figure 1. Overall system architecture of the STM32 multisensor room monitoring system.

Figure 1. Overall system architecture of the STM32 multisensor room monitoring system.

Pin Configuration

Component	STM32 Pin
LDR Analog Output	PA0
DHT22 Data	PA1
Buzzer	PA2
PIR Output	PA3
Encoder CLK	PA4
Encoder DT	PA5
Encoder Button	PB0
OLED SCL	PB6
OLED SDA	PB7
USART1 TX	PA9
USART1 RX	PA10

State Machine

The system has two states. In ACTIVE mode, the OLED, encoder, and alarm behavior are enabled. If no motion is detected for approximately 15 seconds, the system changes to INACTIVE. In INACTIVE mode, the OLED is blanked, encoder input is ignored, and the buzzer is disabled. PIR monitoring remains active so that motion can restore the system to ACTIVE.

State Machine

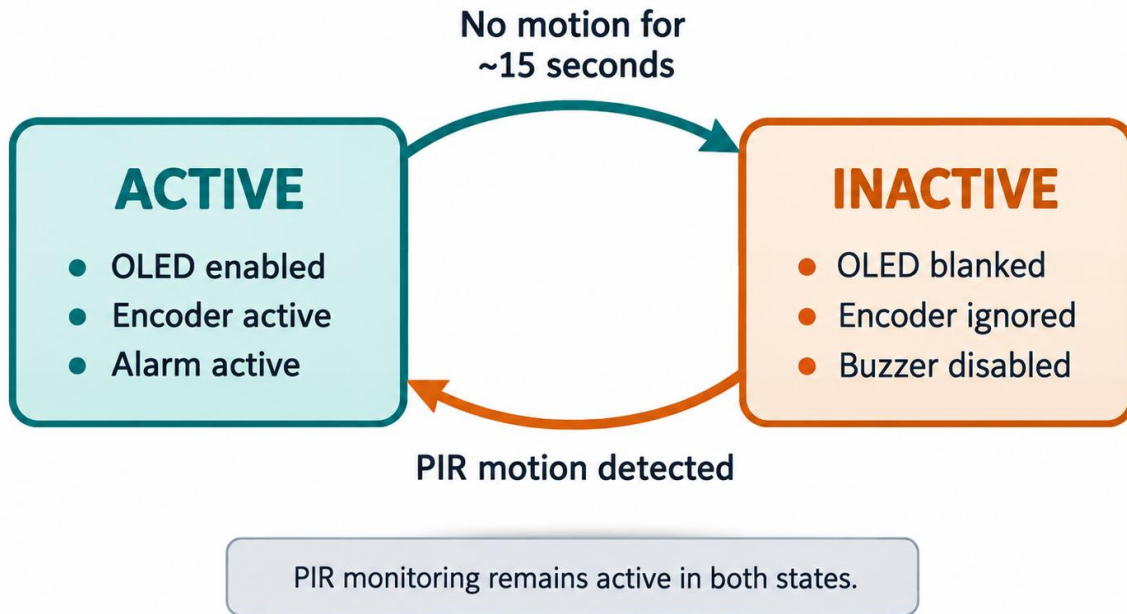


Figure 3. ACTIVE / INACTIVE state machine for the room monitoring system.

Figure 2. ACTIVE/INACTIVE state machine.

3. FreeRTOS Architecture

FreeRTOS was used to separate the application into tasks with different responsibilities and response requirements. Higher-priority tasks are used for motion, user input, and state management, while sensor acquisition and alarm processing use medium priority. OLED updates use the lowest priority because they are less time-critical.

Task	Responsibility	Trigger / Period	Priority	IPC	Typical Blocked Condition
SensorTask	Measurements	2 s	2	Queue	Periodic delay
DisplayTask	OLED update	Event/update	1	Queue / Event Group	Delay / waiting
InputTask	Encoder navigation	Short periodic/event	3	Queue / Event Group	Delay
MotionTask	PIR monitoring	Short periodic	3	Event Group	Delay
AlarmTask	Alarm logic	Sensor update	2	Queue / Event Group	Waiting for data
StateTask	ACTIVE/INACTIVE management	Motion/time event	3	Event Group	Delay

MotionTask, InputTask, and StateTask use priority 3 because motion and user interaction require faster response. SensorTask and AlarmTask use priority 2 because environmental measurements and alarm decisions can tolerate a small scheduling delay. DisplayTask uses priority 1 because OLED updates are less time-critical.

SensorTask uses vTaskDelayUntil() for periodic execution. This is preferred for regular sampling because it uses a reference wake time and reduces timing drift compared with repeatedly delaying relative to the end of the previous execution.

Two sensor queues are used so that DisplayTask and AlarmTask can each receive the latest sensor information independently. The event group represents EVENT_ACTIVE, EVENT_MOTION, and EVENT_ALARM conditions. A UART mutex protects the shared USART1 output so that multiple tasks do not transmit at the same time.

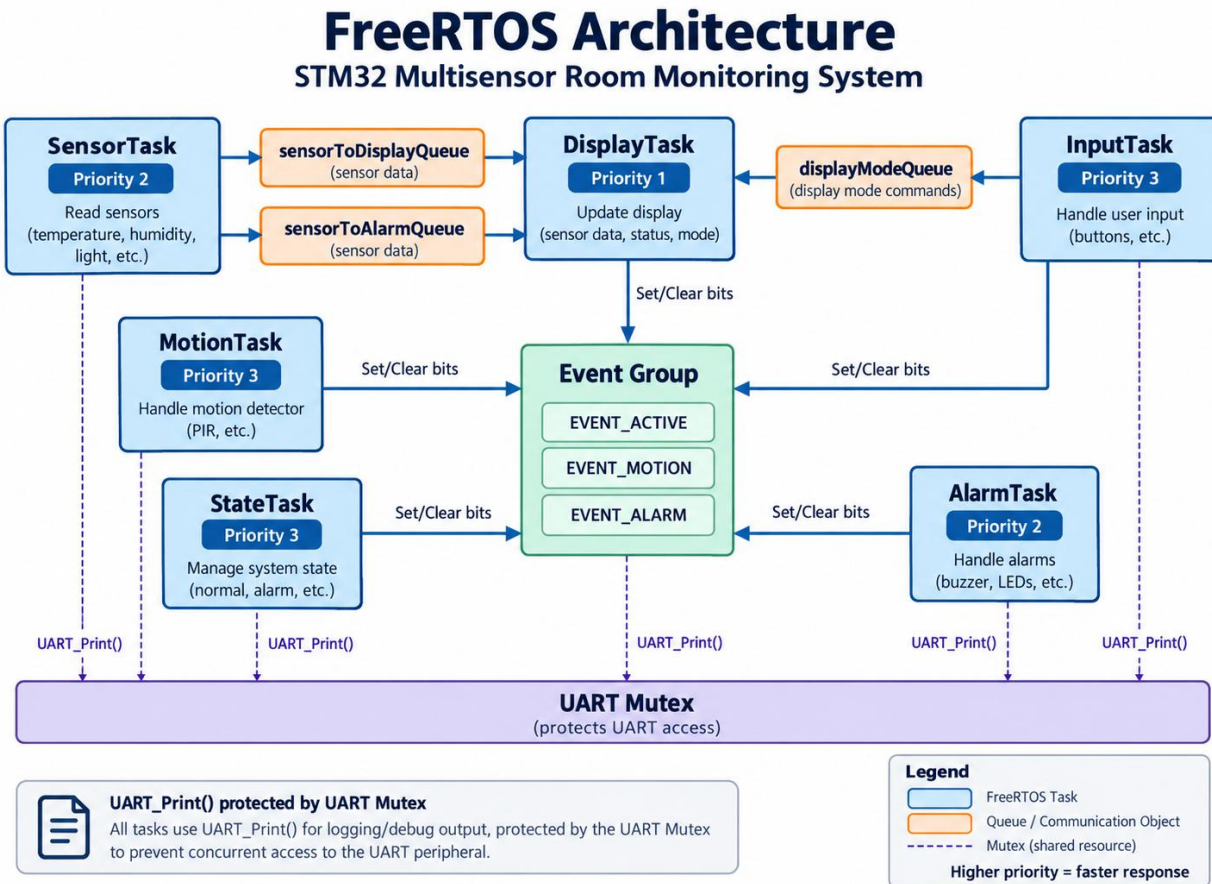


Figure 2. FreeRTOS tasks, priorities, and inter-task communication used in the project.

Figure 3. FreeRTOS tasks, priorities, communication, and synchronization.

4. Implementation

The project was implemented in C/C++ using STM32Cube HAL and native FreeRTOS APIs. The application was divided into multiple source and header files so that main.cpp remained focused on hardware initialization, FreeRTOS object creation, task creation, and scheduler-driven operation.

Temperature alarm logic was separated from buzzer hardware control through evaluateTemperature(). Display navigation was separated through nextDisplayMode() and previousDisplayMode(). System-state behavior was separated through evaluateSystemState(). These functions are deterministic and hardware-independent, which made them suitable for native unit testing.

The rotary encoder uses interrupt-based movement detection to improve responsiveness. InputTask processes the movement and sends the selected display mode through a queue. DisplayTask owns the OLED to avoid unsynchronized writes from multiple tasks.

AlarmTask receives sensor updates through a queue and activates the buzzer only when the temperature is below 18 °C or above 30 °C while the system is ACTIVE. When the system becomes INACTIVE, the buzzer is disabled and the OLED is cleared.

5. Verification and Testing

5.1 Unit Testing

PlatformIO with the Unity test framework was used to test hardware-independent decision logic. A total of 13 meaningful unit tests were implemented and all 13 passed successfully.

Category	Number of Tests	Coverage
Temperature alarm logic	5	Below lower threshold; exact lower threshold; normal value; exact upper threshold; above upper threshold
Display navigation	4	Forward/reverse transitions and wraparound
System state logic	4	ACTIVE/no timeout; ACTIVE/timeout; INACTIVE/no motion; INACTIVE/motion

```

Acey@RM12-SHERIE UCRT64 /c/Users/Acey/bca182-freertos-multisensor
$ ls /c/Users/Acey/bca182-freertos-multisensor/include
FreeRTOSConfig.h README alarm.h app.h buzzer.h display.h input.h system_state.h

Acey@RM12-SHERIE UCRT64 /c/Users/Acey/bca182-freertos-multisensor
$ /c/Users/Acey/.platformio/penv/Scripts/pio.exe test -e test_native
Verbosity level can be increased via '-v, -vv, or -vvv' option
collected 1 tests

Processing test_logic in test_native environment
-----
Building...
Testing...
test\test_logic\test_main.cpp:221: test_temperature_below_lower_limit [PASSED]
test\test_logic\test_main.cpp:222: test_temperature_exact_lower_limit [PASSED]
test\test_logic\test_main.cpp:223: test_temperature_normal_value [PASSED]
test\test_logic\test_main.cpp:224: test_temperature_exact_upper_limit [PASSED]
test\test_logic\test_main.cpp:225: test_temperature_above_upper_limit [PASSED]
test\test_logic\test_main.cpp:227: test_navigation_forward [PASSED]
test\test_logic\test_main.cpp:228: test_navigation_forward_wraparound [PASSED]
test\test_logic\test_main.cpp:229: test_navigation_reverse [PASSED]
test\test_logic\test_main.cpp:230: test_navigation_reverse_wraparound [PASSED]
test\test_logic\test_main.cpp:232: test_active_without_timeout [PASSED]
test\test_logic\test_main.cpp:233: test_active_with_timeout [PASSED]
test\test_logic\test_main.cpp:234: test_inactive_without_motion [PASSED]
test\test_logic\test_main.cpp:235: test_inactive_with_motion [PASSED]
----- test_native:test_logic [PASSED] Took 4.01 seconds -----

===== SUMMARY =====
Environment  Test      Status  Duration
-----
test_native  test_logic PASSED  00:00:04.015
===== 13 test cases: 13 succeeded in 00:00:04.015 =====

Acey@RM12-SHERIE UCRT64 /c/Users/Acey/bca182-freertos-multisensor
$

```

Figure 4. PlatformIO unit-test result showing 13 tests succeeded.

5.2 Wokwi Functional Verification

Test ID	Stimulus	Expected	Observed	Result
FT-01	Change temperature	Displayed temperature updates	Displayed/serial temperature changed	PASS
FT-02	Change humidity	Displayed humidity updates	Humidity value changed	PASS
FT-03	Change light input	Light value changes	Light value changed	PASS

FT-04	Rotate encoder clockwise	Next page selected	Next page selected	PASS
FT-05	Rotate encoder counterclockwise	Previous page selected	Previous page selected	PASS
FT-06	Set temperature above 30 °C	Alarm activates	Buzzer/high alarm activated	PASS
FT-07	Return temperature to normal	Alarm stops	Buzzer stopped / NORMAL	PASS
FT-08	Trigger PIR	System is ACTIVE	Motion detected / ACTIVE	PASS
FT-09	Allow inactivity timeout	System becomes INACTIVE	INACTIVE after timeout	PASS
FT-10	Trigger PIR while INACTIVE	System returns to ACTIVE	ACTIVE restored	PASS

5.3 Deliberate Fault Experiments

The laboratory handout also requires deliberate experiments involving removal of blocking, changing task priority, and temporarily removing the UART mutex. These experiments were not completed before this report draft was generated. The final verified implementation keeps task blocking, intended priorities, and UART mutex protection enabled. If required for final submission, the three experiments should still be performed and documented with observations before submitting the final PDF.

6. Static Code Analysis

Static code analysis was performed with PlatformIO using the command `pio check -e bluepill_f103c8`. The analysis completed successfully for the STM32 Blue Pill environment. No significant project-source warning requiring correction was reported in the final run.

Finding	File / Line	Cause	Resolution
No significant findings	N/A	No major cppcheck issues reported	No corrective action required

A successful static-analysis result does not guarantee that the program is free of all defects, but it indicates that the enabled checks did not identify major issues in the project source code.

7. Engineering Discussion

One important design decision was the use of two sensor queues. If DisplayTask and AlarmTask consumed from only one queue, one task could remove data before the other received it. Separate queues allow both consumers to receive updated sensor information independently.

Another decision was assigning OLED ownership to DisplayTask. This avoids unsynchronized display access from multiple tasks and keeps display logic in one location. UART output was also treated as a shared resource and protected with a mutex.

During development, some application header files were initially placed inside the FreeRTOS library include directory. They were later moved to the project-level include directory to keep application code separate from third-party library files. Native unit testing also required correct include paths and a working native C++ compiler before the tests could run successfully.

The Wokwi simulation depends on the generated firmware.elf file, so the STM32 environment must be built before starting the simulation. This was an important practical debugging lesson during the activity.

The current implementation is mainly verified in simulation. Physical hardware may introduce sensor noise, electrical differences, timing issues, and additional encoder debouncing requirements. The LDR reading is

a relative light level rather than calibrated lux, and the approximately 15-second inactivity timeout is intentionally short for demonstration.

8. Conclusion

This laboratory activity demonstrated the design of a concurrent embedded system using the STM32 Blue Pill and FreeRTOS. The completed project monitors temperature, humidity, ambient light, and motion while providing OLED navigation, temperature-alarm behavior, and automatic ACTIVE/INACTIVE state transitions.

The project provided practical experience with task creation, task priorities, queues, event groups, mutex protection, periodic execution using `vTaskDelayUntil()`, and state-machine design. Separating hardware-independent logic also made it possible to perform native automated testing, where all 13 unit tests passed.

Future improvements could include physical-hardware testing, better rotary-encoder debouncing, calibrated light measurement, configurable temperature thresholds, data logging, wireless communication, and more advanced low-power operation.

Requirements Traceability Matrix

Requirement	Implementation	Verification
FR-01	SensorTask	FT-01
FR-02	SensorTask	FT-02
FR-03	SensorTask	FT-03
FR-04	MotionTask	FT-08
FR-05	DisplayTask	FT-01 to FT-03
FR-06	InputTask	Unit navigation tests + FT-04/FT-05
FR-07	AlarmTask	Unit alarm tests + FT-06/FT-07
FR-08	StateTask	Unit state tests + FT-08 to FT-10
FR-09	MotionTask / StateTask	Unit state tests + FT-09
FR-10	MotionTask / StateTask	Unit state tests + FT-10

References

1. BCA182 Embedded Systems Programming - Laboratory Activity No. 1: Real-Time Multisensor Room Monitoring System, September 2026.
2. FreeRTOS Kernel documentation.
3. STM32Cube HAL documentation.
4. PlatformIO documentation.
5. Wokwi simulator documentation.